- **赋值运算符**
  - 赋值运算符：对变量进行赋值的运算符
    - 已经学过的赋值运算符：=
    - 其他赋值运算符：
      - +=
      - -=
      - *=
      - /=
      - %=
  - 使用这些运算符可以在对变量赋值时进行快速操作
  - 我们以 += 赋值运算符来举例
    - 以前我们让一个变量加 1 如何做的?

```
<script>
    let num = 1
    num = num + 1
    console.log(num)  // 结果是 2
</script>
```

    - 现在我们有一个简单的写法啦~~~

```
<script>
    let num = 1
    num += 1
    console.log(num)  // 结果是 2
</script>
```

- **一元运算符**
  - 二元运算符：需要两个表达式才能进行运算

```
let num = 10 + 20
```

  - 三元运算符：需要三个表达式才能进行运算

```
let max = 10 > 5 ? 10 : 5
```

  - 一元运算符：需要一个表达式就可以进行运算

```
let res = !true
```

  - 自增：
    - 符号：++
    - 作用：让变量的值 +1
  - 自减：
    - 符号：--
    - 作用：让变量的值 -1
  - 使用场景：

        - 经常用于计数来使用。 比如进行10次操作，用它来计算进行了多少次了
    - 自增运算符的用法：
        - 前置自增：

```
let num = 1
++num      //让num的值加 1变 2
```

            - 每执行1次，当前变量数值加1
            - 其作用相当于 num += 1
        - 后置自增：

```
let num = 1
num++      //让num的值加 1变 2
```

            - 每执行1次，当前变量数值加1
            - 其作用相当于 num += 1
        - 前置自增和后置自增如果参与运算就有区别：
            - 前置自增：先自加再运算

```
let i = 1
console.log(++i + 2)  //结果是 4
//  注意：i是 2
//  i先自加 1，变成2之后，在和后面的2相加
```

            - 后置自增：先运算再自加

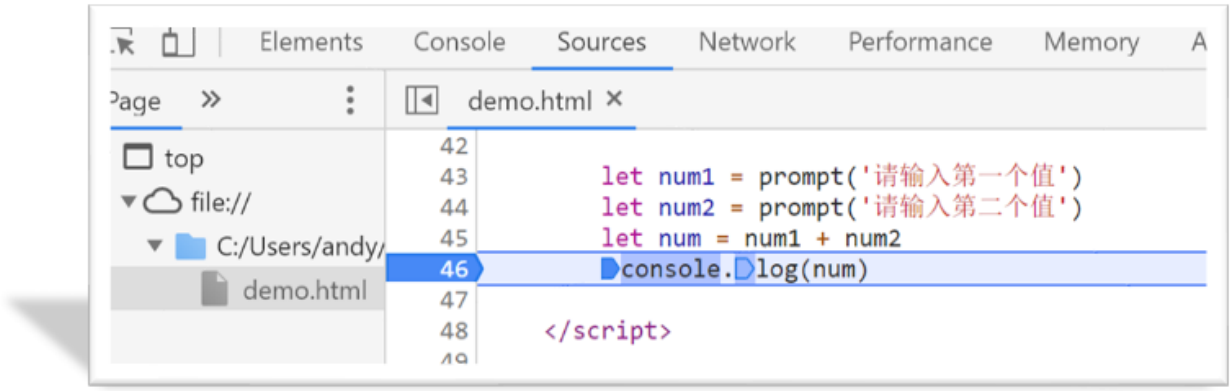
```
let i = 1
console.log(i++ + 2)  //结果是 3
// 注意：  此时的 i是 1
// 先和2相加，先运算输出完毕后，i再自加是2
```

- **运算符优先级**

优先级	运算符	顺序
1	小括号	()
2	一元运算符	!
3	算数运算符	**先 * / % 后 + -**
4	关系运算符	> >= < <=
5	相等运算符	== != === !==
6	逻辑运算符	**先 && 后 ||**
7	赋值运算符	=
8	逗号运算符	,

    - 一元运算符里面的逻辑非优先级很高
    - 逻辑与比逻辑或优先级高

- **断点调试**
  - 作用：学习时可以帮助更好的理解代码运行，工作时可以更快找到bug
  - 浏览器打开调试界面
    - 按F12打开开发者工具
    - 点到sources一栏
    - 选择代码文件
  - 断点：在某句代码上加的标记就叫断点，当程序执行到这句有标记的代码时会暂停下来

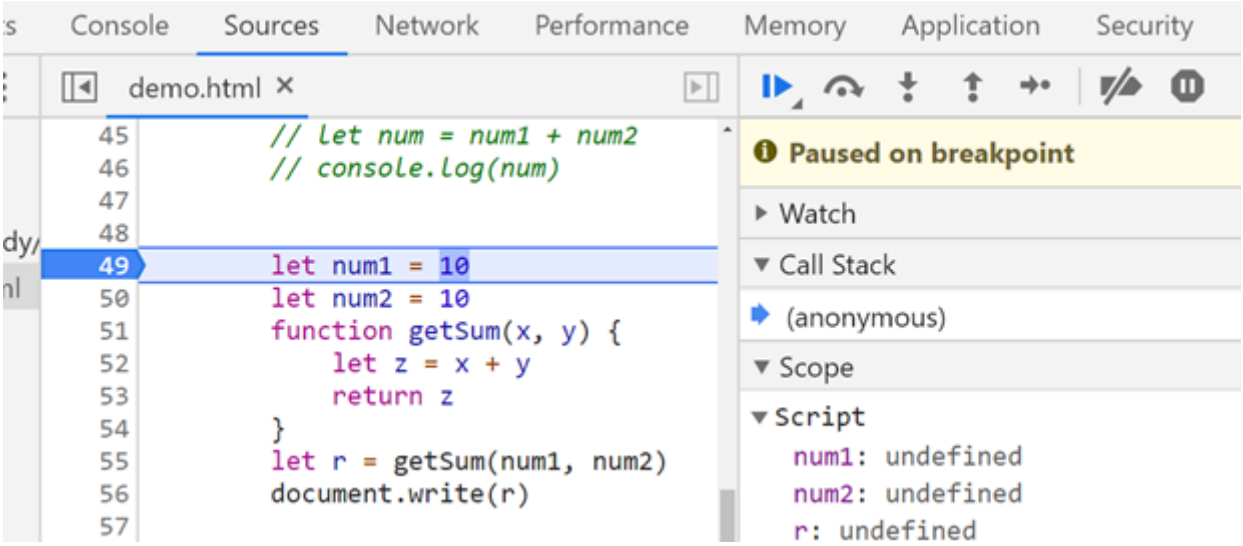


按钮1：继续执行下面的代码，直到遇到下个断点(如果没有下个断点就执行代码到结束)

按钮2：点一次执行一句代码，如果遇到函数，一次性把函数执行完毕

按钮3：点一次执行一句代码，如果遇到函数，会进入函数内：点一次，执行一次代码

按钮4：如果此时在函数内，则一次性执行函数内剩余的代码



- **while 循环**
  - while 循环语法：

```
while (循环条件) {
    要重复执行的代码(循环体)
}
```

- 释义：
  - 跟if语句很像，都要满足小括号里的条件为true才会进入执行代码
  - while大括号里代码执行完毕后不会跳出，而是继续回到小括号里判断条件是否满足，若满足又执行大括号里的代码，然后再回到小括号判断条件，直到括号内条件不满足，即跳出
- while 循环注意事项：
  - 循环的本质就是以某个变量为起始值，然后不断产生变化量，慢慢靠近终止条件的过程。所以，循环需要具备三要素：

1. 循环变量起始值
2. 终止条件（没有终止条件，循环会一直执行，造成死循环）
3. 循环变量的改变（增加或者减少）

```
let i = 1
while (i <= 3) {
    document.write('我会循环三次<br>')
    i++
}
```

- **do-while 循环**
  - do-while 循环语法：

```
do {
    要重复执行的代码(循环体)
} while (循环条件)
```

- 释义：
  - 先执行do 里面的语句
  - 执行完毕后，到 while里面判断条件是否满足，如果满足则回头继续执行do里面的语句，如果条件不满足，则退出循环

- **for 循环**
  - for循环语法

```
for (声明记录循环次数的变量; 循环条件; 变化值) {
    循环体
}
```

- 也是重复执行代码
- 好处：把声明起始值、循环条件、变化值写到一起，让人一目了然

- **三种循环区别**

- 三种循环区别：
  - for 语句和 while 语句是完全可互换的，for 的语法结构更简洁，**故 for 循环的使用频次会更多**
  - 如果涉及到**条件判断**的时候使用while 更合适
  - 如果涉及到**计算次数**的时候使用for更简单，语法更简单明了
  - do-while 循环体至少使用一次如果某些代码一定会执行一次可以考虑

```
let love = prompt('你爱我吗')

while (love !== '爱') {
    love = prompt('你爱我吗')
}
alert('我也爱你')
```

```
for (let i = 1; i <= 5; i++) {

    document.write(`★`)
}
```

- **循环退出**
  - continue：结束本次循环，继续下次循环
  - break：跳出所在的循环